República Bolivariana de Venezuela

Ministerio del Poder Popular para la Educación Universitaria

Universidad Politécnica Territorial

´´José Antonio Anzoátegui´´

Trayecto 1 Fase 2 PNF: Informática

Municipio Guanta, Edo. Anzoátegui

<Punteros>

Profesora:

Lennis Humner

Estudiantes:

Rubén Díaz

C.L: 32.500.566

Guanta, 23 de marzo de 2026

❖ Definición

Un puntero es, en esencia, una variable cuyo valor no es un dato convencional (como un número o una letra), sino una dirección de memoria.

Imagina que la memoria de tu computadora es un hotel gigante con miles de habitaciones numeradas. Si una variable normal es el huésped que vive dentro de la habitación, un puntero es un papelito que tiene escrito el número de esa habitación.

✓ Conceptos Clave para Entenderlo

Para manejar punteros, necesitas dominar dos operadores fundamentales (comunes en lenguajes como C o C++):

1. **Operador de dirección (&):** Se usa para obtener la ubicación de una variable. Si tienes x, &x te dice en qué "habitación" vive x.

2. **Operador de desreferenciación (*):** Se usa para acceder al contenido de la dirección que guarda el puntero. Es como decir: "ve a la dirección que dice este papel y dime qué hay dentro".

❖ Declaración:

La declaración de un puntero es el proceso de crear una variable que, en lugar de almacenar un dato directo, reserva espacio para guardar una dirección de memoria.

Para declarar un puntero en lenguajes como C o C++, se utiliza el operador asterisco (*). La sintaxis básica es:

tipo_de_dato *nombre_del_puntero;

---

1. Elementos de la declaración

- **Tipo de dato:** Indica qué tipo de contenido habrá en la dirección a la que apunta (int, float, char, etc.). Es crucial porque el procesador necesita saber cuántos bytes leer en esa dirección.
- **El asterisco (*):** Es el símbolo que le dice al compilador: "Esta variable no es un número, es un puntero".
- **Nombre:** El identificador que usaremos para referirnos al puntero.

---

2. Ejemplos comunes de declaración

Dependiendo de lo que quieras apuntar, la declaración cambia ligeramente:

Declaración	Descripción
int *p;	Un puntero que apuntará a un número entero.
char *letra;	Un puntero que apuntará a un carácter (o al inicio de una cadena).
float *promedio;	Un puntero que apuntará a un número con decimales.
void *generico;	Un puntero "comodín" que puede apuntar a cualquier tipo de dato.

3. Inicialización (Paso crítico)

Declarar un puntero no es suficiente; siempre debes inicializarlo. Un puntero declarado pero no asignado apunta a una dirección aleatoria ("basura"), lo que puede hacer que tu programa falle (Crash).

Hay dos formas principales de inicializarlo:

A**. Apuntando a una variable existente:**

Usamos el operador de dirección (&):

C:

```
int meta = 100;
int *pPtr = &meta; // pPtr ahora guarda la dirección de 'meta'
```

B. Inicializando en NULL

Si no vas a usar el puntero de inmediato, es una buena práctica asignarle NULL para evitar que apunte a lugares peligrosos:

C:

```
int *pPtr = NULL;
```

---

4. ¿Dónde va el asterisco?

En programación, la gente escribe el asterisco de distintas formas. Todas son válidas para el compilador, pero la primera es la más común entre profesionales:

- int *p; (Preferido: el puntero es p)
- int* p; (Enfatiza que el tipo es "puntero a entero")
- int * p; (Menos común)

❖ Operadores:

1. El Operador de Dirección (&)

Este operador se conoce como referencia. Se coloca delante del nombre de una variable para obtener su ubicación física en la memoria RAM, en lugar de su valor.

- **Función:** "Dime en qué dirección vive esta variable".
- **Resultado:** Una dirección de memoria (usualmente un número hexadecimal como 0x7ffce).

Ejemplo:

C:

```
int edad = 25;
printf("%p", &edad); // Imprime la dirección de memoria de 'edad'
```

---

2. El Operador de Indirección o Desreferencia (*)

Este operador tiene un doble uso dependiendo de dónde lo coloques, lo que suele causar confusión al principio:

A. En la Declaración

Se usa para avisar al compilador que la variable que estás creando es un puntero.

C:

```
int *p; // Aquí '*' significa: "p es un puntero que guardará una dirección"
```

B. En el Uso (Acceso)

Cuando se usa con un puntero que ya tiene una dirección guardada, sirve para acceder al contenido de esa dirección.

- **Función:** "Ve a la dirección que tiene guardada este puntero y dime qué hay dentro (o modifícalo)".

Ejemplo:

C:

```
int numero = 10;
int *p = &numero; // 'p' guarda la dirección de 'numero'

printf("%d", *p);  // Imprime 10 (el contenido de la dirección en p)
*p = 50;           // Cambia el valor de 'numero' a 50 a través del puntero
```

Resumen de Operadores

Operador	Nombre	Acción
&	Referencia	Obtiene la **dirección** de una variable.
*****	Desreferencia	Accede al **valor** almacenado en una dirección.

❖ Operaciones

1. Suma y Resta de Enteros

Cuando sumas un entero n a un puntero, no le sumas n bytes, sino n posiciones de memoria del tipo de dato correspondiente.

- **Si es un int (4 bytes):** Al hacer p + 1, la dirección aumenta 4 bytes.

- **Si es un char (1 byte):** Al hacer p + 1, la dirección aumenta 1 byte.

Uso principal: Recorrer arreglos de forma eficiente sin usar índices [i].

---

2. Incremento (++) e Decremento (--)

Son las operaciones más comunes. Mueven el puntero a la siguiente o anterior casilla de memoria del mismo tipo.

C:

```
int numeros[] = {10, 20, 30};
int *p = numeros; // p apunta al 10

p++; // Ahora p apunta al 20 (saltó 4 bytes en memoria)
```

---

3. Resta de dos Punteros

Puedes restar dos punteros siempre que apunten al mismo arreglo. El resultado no es la distancia en bytes, sino el número de elementos que hay entre ellos.

- **Ejemplo:** Si p2 apunta al elemento 5 y p1 al elemento 2, p2 - p1 dará como resultado 3.

---

4. Comparación de Punteros

Puedes usar operadores relacionales (==, !=, <, >, <=, >=) para comparar dos punteros. Esto es útil para saber:

- Si dos punteros apuntan a la misma variable.
- Cuál elemento está "antes" o "después" dentro de un mismo bloque de memoria (como un vector).

Resumen de Operaciones Prohibidas:

Para evitar errores graves de memoria, nunca intentes:

- **Sumar dos punteros:** No tiene sentido lógico sumar dos direcciones de memoria.
- **Multiplicar o dividir punteros:** Estas operaciones no están definidas.
- **Sumar decimales:** Solo se pueden sumar o restar números enteros.

Tabla de Referencia Rápida

Operación	**Sintaxis**	**Resultado**
Asignación	p1 = p2;	Ambos apuntan al mismo lugar.
Suma	p + n	Avanza n elementos en la memoria.
Resta	p1 - p2	Distancia en número de elementos.
Comparación	p1 < p2	Determina posición relativa.

❖ Funciones

1. Paso de parámetros por Referencia

Normalmente, cuando pasas una variable a una función, se crea una copia de su valor. Si modificas esa copia dentro de la función, la variable original fuera de ella no cambia.

Al usar punteros, pasas la dirección de memoria. Esto permite que la función entre a esa dirección y modifique el dato original directamente.

¿Por qué usarlo?

- Para que una función pueda "devolver" más de un valor modificando las variables originales.
- Para ahorrar memoria y tiempo al no tener que copiar estructuras de datos muy grandes.

Ejemplo en C:

C:
```
void duplicar(int *p) {
    *p = (*p) * 2; // Accedemos al valor original y lo duplicamos
}

int main() {
    int n = 10;
    duplicar(&n); // Pasamos la dirección de 'n'
    // Ahora 'n' vale 20
}
```

---

2. Funciones que devuelven Punteros

Una función puede devolver una dirección de memoria. Esto es común cuando la función reserva memoria de forma dinámica o cuando busca un elemento dentro de una estructura (como un arreglo) y devuelve su ubicación.

Regla de oro: Nunca devuelvas un puntero a una variable local de la función. Las variables locales desaparecen cuando la función termina, dejando al puntero apuntando a "basura".

Ejemplo correcto (Memoria Dinámica):

C:
```
int* crearArreglo(int tam) {
   int *p = malloc(tam * sizeof(int)); // Pedimos memoria al sistema
   return p; // Devolvemos la dirección del nuevo bloque
}
```

---

3. Punteros a Funciones

Aunque suena avanzado, en C y C++ puedes tener un puntero que no apunta a un dato, sino al **código de una función**. Esto permite pasar funciones como si fueran variables.

- **Utilidad:** Se usa para crear "callbacks" (funciones que se ejecutan después de un evento) o para implementar polimorfismo de bajo nivel.

4. Sintaxis de declaración:

tipo_retorno (*nombre_puntero)(tipo_parametros);

❖ Estructuras

1. Punteros a Estructuras

Al igual que con los enteros o caracteres, puedes declarar un puntero que apunte a una estructura completa. Esto es vital cuando pasas registros pesados a funciones, ya que solo pasas la dirección de memoria (unos pocos bytes) en lugar de copiar toda la información de la estructura.

C:

```
struct Persona {
   char nombre[50];
   int edad;
};

struct Persona usuario;
struct Persona *pPtr = &usuario; // El puntero apunta a la estructura 'usuario'
```

2. El Operador Flecha (->)

Esta es la operación más importante cuando trabajas con punteros y estructuras. Para acceder a los miembros de una estructura a través de un puntero, se utiliza -> en lugar del punto (.).

- **usuario.edad**: Se usa cuando tienes la variable directa.
- **pPtr->edad**: Se usa cuando tienes un puntero a la estructura.

---

3. Estructuras Autorreferenciadas

Una estructura es autorreferenciada cuando uno de sus miembros es un **puntero que apunta al mismo tipo de estructura**. Este es el bloque de construcción básico para las **Estructuras de Datos Dinámicas**.

Aplicaciones principales:

- **Listas Enlazadas:** Cada "nodo" tiene datos y un puntero al "siguiente" nodo.
- **Árboles Binarios:** Cada nodo tiene punteros a un "hijo izquierdo" y un "hijo derecho".
- **Pilas y Colas:** Estructuras que crecen y decrecen según la necesidad del programa.

---

4. Ejemplo de Nodo (Lista Enlazada)

Para que visualices cómo se ve esto en código, así se define un nodo básico que podría usarse en un sistema de gestión cultural o administrativo:

C:

```
struct Nodo {
   int id;              // Dato que guardamos
   struct Nodo *sig;    // Puntero al siguiente elemento
};

struct Nodo *inicio = NULL; // Puntero que marca el comienzo de la lista
```

Ventajas de usar Punteros con Estructuras

1. **Memoria Dinámica:** Puedes crear nuevos registros (nodos) en tiempo de ejecución usando malloc.
2. **Organización:** Permite conectar datos que están en lugares distintos de la memoria RAM como si fueran una cadena.
3. **Velocidad:** Modificar un puntero para que apunte a otro nodo es mucho más rápido que mover bloques enteros de datos.

➢ Listas Enlazadas

❖ Concepto

Una lista enlazada es una estructura de datos dinámica y lineal en la que los elementos, llamados nodos, no se guardan en posiciones contiguas de la memoria (como ocurre en un arreglo), sino que se dispersan y se conectan entre sí mediante punteros.

Es como una cadena humana donde cada persona tiene un objeto y, con una mano, señala a la siguiente persona de la fila.

---

1. Anatomía de un Nodo

Para que una lista funcione, cada nodo debe tener al menos dos partes:

1. **Dato:** La información que queremos almacenar (un número, un nombre, un objeto complejo).
2. **Enlace (Puntero):** Una variable que guarda la dirección de memoria del siguiente nodo en la lista.

2. Elementos de Control

Para gestionar una lista enlazada, necesitamos dos punteros especiales:

- **Cabeza (Head):** Un puntero que apunta al primer nodo. Si la cabeza es NULL, la lista está vacía.
- **Cola (Tail):** Opcionalmente, un puntero al último nodo para facilitar inserciones rápidas al final.
- **Nodo Final:** El último nodo de la lista siempre apunta a NULL, indicando que ya no hay más elementos.

---

3. Diferencias con los Arreglos (Arrays)

Característica	Arreglo (Array)	Lista Enlazada
Tamaño	Fijo (estático).	Dinámico (crece según se necesite).
Memoria	Bloque contiguo (todo	Esparcida por la memoria

Característica	Arreglo (Array)	Lista Enlazada
	junto).	RAM.
Inserción	Costosa (hay que mover elementos).	Muy rápida (solo cambias un puntero).
Acceso	Directo (por índice [i]).	Secuencial (hay que recorrer desde el inicio).

---

4. Tipos de Listas Enlazadas

Existen variaciones según cómo se conecten los nodos:

1. **Simplemente enlazada:** Cada nodo apunta solo al siguiente.
2. **Doblemente enlazada:** Cada nodo apunta al siguiente y al anterior (permite recorrer en ambos sentidos).
3. **Circular:** El último nodo apunta de nuevo al primero, formando un ciclo.

❖ Clasificación

1. Lista Simplemente Enlazada (Singly Linked List)

Es la variante más básica. Cada nodo contiene el dato y un único puntero que apunta al siguiente nodo.

- **Recorrido:** Solo hacia adelante (unidireccional).
- **Final:** El último nodo apunta a NULL.
- **Uso:** Ideal para estructuras simples donde la memoria es limitada y solo necesitas avanzar.

2. Lista Doblemente Enlazada (Doubly Linked List)

Cada nodo tiene dos punteros: uno al nodo siguiente y otro al nodo anterior.

- **Recorrido:** Bidireccional (puedes ir hacia adelante y hacia atrás).
- **Ventaja:** Facilita mucho la eliminación de nodos, ya que desde un nodo puedes saber quién es su predecesor sin tener que volver a recorrer toda la lista desde el inicio.
- **Costo:** Consume un poco más de memoria por el puntero extra.

---

3. Lista Circular Simplemente Enlazada

Es similar a la simple, con la diferencia de que el último nodo no apunta a NULL, sino que apunta de nuevo al primer nodo (la cabeza).

- **Estructura:** Forma un ciclo infinito.
- **Uso:** Muy útil en sistemas que deben repartir recursos de forma equitativa (Round Robin), como el tiempo de CPU entre varios procesos de un sistema operativo.

---

4. Lista Circular Doblemente Enlazada

Es la combinación de las dos anteriores. El último nodo apunta al primero, y el primero tiene su puntero "anterior" apuntando al último.

- **Versatilidad:** Es la más flexible de todas. Permite dar vueltas completas a la lista en cualquier dirección.
- **Uso:** Estructuras de datos complejas como buffers de reproducción de música o gestión de pestañas en un navegador.

❖ Listas simplemente enlazadas: Fundamentos Teóricos

1. El Concepto de Nodo

El nodo es la unidad básica e indivisible de la lista. Desde el punto de vista teórico, un nodo es una estructura que contiene dos campos bien diferenciados:

- **Campo de Información (Dato):** Almacena el valor o la entidad que nos interesa (un entero, un objeto, una estructura).
- **Campo de Enlace (Siguiente):** Es un puntero que almacena la **dirección de memoria** del próximo nodo en la secuencia.

2. Estructura y Terminología

Para que una lista sea accesible y manipulable, existen elementos de control que definen su estado:

- **Cabeza (Head):** Es un puntero externo que apunta al primer nodo de la lista. Si la Cabeza es igual a NULL, se dice que la **lista está vacía**.
- **Cuerpo:** Son todos los nodos intermedios que conectan la cabeza con el final.
- **Fin de Lista:** Se reconoce porque el campo de enlace del último nodo apunta a una dirección nula (NULL), indicando que no hay más elementos que procesar.

---

3. Propiedades Dinámicas

A diferencia de las estructuras estáticas, las listas simplemente enlazadas poseen propiedades únicas:

- **Asignación Dinámica:** La memoria se solicita al sistema operativo en tiempo de ejecución (usando funciones como malloc en C). Esto significa que la lista crece o se encoge según la necesidad real del programa.
- **No Contigüidad:** Los nodos no ocupan posiciones consecutivas en la memoria física. El orden lógico lo imponen los punteros, no la ubicación física en la RAM.

- **Acceso Secuencial:** Para llegar al quinto elemento, obligatoriamente debes pasar por el primero, segundo, tercero y cuarto. No existe el "acceso aleatorio" por índice como en los arreglos.

---

4. Operaciones Fundamentales

La teoría de listas define un conjunto de operaciones básicas que mantienen la integridad de la estructura:

1. **Inserción:** Puede realizarse al inicio, al final o entre dos nodos existentes. Requiere "re-apuntar" los enlaces para no perder la conexión de la cadena.
2. **Eliminación:** Consiste en sacar un nodo de la secuencia. Es vital liberar la memoria del nodo eliminado para evitar **fugas de memoria** (*memory leaks*).
3. **Recorrido (Traversado):** Visitar cada nodo partiendo desde la Cabeza hasta encontrar el valor NULL.
4. **Búsqueda:** Recorrer la lista comparando el campo de información de cada nodo con un valor específico.

❖ Listas simplementes enlazadas: clasificación

1. Según la existencia de un Nodo Cabecera (Header Node)

Esta clasificación se basa en cómo comienza la lista en la memoria:

- **Lista con Nodo Cabecera:** Existe un nodo especial al principio que no contiene datos útiles (o contiene metadatos como el tamaño de la lista). Su función es simplificar las operaciones de inserción y borrado, ya que la "cabeza" de la lista nunca cambia de dirección; siempre apunta al nodo cabecera.
- **Lista sin Nodo Cabecera:** La lista comienza directamente con el primer nodo de datos. Es la forma más eficiente en uso de memoria, pero requiere un cuidado especial al

insertar o eliminar el primer elemento, ya que el puntero inicio debe actualizarse.

---

2. Según el Orden de los Elementos

Determina cómo se disponen los nodos dentro de la secuencia:

- **Listas No Ordenadas:** Los elementos se insertan en cualquier posición (generalmente al principio o al final) sin seguir un criterio de magnitud. Es ideal para pilas (LIFO) o colas (FIFO) simples.
- **Listas Ordenadas:** Cada vez que se inserta un nuevo nodo, el algoritmo busca su posición correcta (de menor a mayor o viceversa). Esto hace que la inserción sea más lenta ($O(n)$), pero las búsquedas y la generación de reportes sean más rápidas.

---

3. Según su Comportamiento Lógico (Tipos de TAD)

Las listas simplemente enlazadas son la base para implementar **Tipos Abstractos de Datos (TAD)** específicos:

- **Pilas (Stacks):** El último elemento en entrar es el primero en salir (LIFO). Las inserciones y extracciones se realizan siempre por el mismo extremo (la cabeza).
- **Colas (Queues):** El primero en entrar es el primero en salir (FIFO). Requiere un puntero al inicio para extraer y un puntero al final para insertar.

---

4. Según la Finalización del Enlace

Aunque estrictamente una lista "circular" se considera una categoría aparte, en teoría de algoritmos se clasifica como una variante de la simple:

- **Lineales:** El último nodo apunta a NULL. Tienen un principio y un fin definidos.
- **Circulares Simples:** El último nodo guarda la dirección del primero. No tienen un final físico, lo que permite recorridos infinitos (útil para buffers de datos o turnos de procesos).

❖ Operaciones básicos: Declaración

1. Definición de la Estructura del Nodo

Un nodo es una estructura compuesta (usualmente un struct en lenguajes como C o C++). Debe contener al menos dos elementos:

- **El Dato:** La información que queremos guardar (puede ser un entero, una cadena de texto o incluso otra estructura).
- **El Enlace:** Un puntero del **mismo tipo** que la estructura que estamos definiendo.

Ejemplo en C:

C:
```
struct Nodo {
   int dato;            // Información del nodo
   struct Nodo *siguiente; // Puntero al próximo nodo
};
```

---

2. Declaración del Puntero de Control (Cabeza)

Una vez definida la "forma" del nodo, necesitamos una variable que represente a la lista completa. Esta variable es un puntero que siempre apuntará al **primer elemento**.

- **Nombre común:** inicio, cabecera o head.
- **Estado inicial:** Siempre debe inicializarse en NULL para indicar que la lista está actualmente vacía.

C
```
struct Nodo *inicio = NULL;
```

---

3. Creación de Nuevos Nodos (Instanciación)

La declaración por sí sola no crea elementos en la memoria. Para agregar datos, debemos solicitar espacio de forma dinámica durante la ejecución del programa.

En C, esto se hace mediante la función malloc (memory allocation), la cual reserva el tamaño exacto que ocupa nuestra estructura en la memoria RAM.

Proceso lógico de creación:

1. Se declara un puntero auxiliar (ej. nuevo).
2. Se le asigna memoria: nuevo = (struct Nodo *)malloc(sizeof(struct Nodo));.
3. Se le asigna el dato: nuevo->dato = 100;.
4. Se asegura que su enlace sea NULL (si es el último) o que apunte al siguiente nodo.

❖ Operaciones básicos: Punteros de cabecera y cola

1. El Puntero de Cabecera (Head)

Es el punto de entrada obligatorio. Sin él, la lista se pierde en la memoria RAM (lo que causaría un *memory leak*).

- **Propósito:** Almacenar la dirección del primer nodo.
- **Operación clave:** Si quieres recorrer la lista o insertar al principio, usas este puntero.
- **Estado vacío:** head = NULL.

2. El Puntero de Cola (Tail)

Es un puntero adicional que siempre apunta al **último nodo** de la lista (aquel cuyo siguiente es NULL).

- **Propósito:** Evitar tener que recorrer toda la lista (desde el primero hasta el último) para añadir un elemento al final.
- **Eficiencia:** Reduce la complejidad de inserción al final de **$O(n)$** (lento) a **$O(1)$** (instantáneo).
- **Estado vacío:** tail = NULL.

---

3. Declaración y Estructura en Código

Para implementar esto, usualmente se define una estructura que agrupe ambos punteros de control. Esto mantiene el código organizado, especialmente en proyectos grandes de sistemas administrativos o de gestión.

C:
```
// Definición del nodo
struct Nodo {
   int dato;
   struct Nodo *siguiente;
};

// Estructura de control para la lista
```

```
struct Lista {
   struct Nodo *cabecera; // Apunta al primero
   struct Nodo *cola;     // Apunta al último
};

// Inicialización
struct Lista miLista;
miLista.cabecera = NULL;
miLista.cola = NULL;
```

❖ Operaciones básicos: Operador de selección

1. El Operador Punto (.)

Se utiliza cuando trabajas con una instancia directa de la estructura (una variable estática).

- **Uso:** variable.miembro
- **Ejemplo:** Si declaras struct Nodo miNodo;, accedes a su dato con miNodo.dato.

2. El Operador Flecha (->)

Este es el verdadero protagonista cuando manejas listas enlazadas. Se utiliza exclusivamente cuando tienes un puntero que apunta a una estructura.

- **Uso:** puntero->miembro
- **Definición técnica:** Es una forma abreviada de escribir (*puntero).miembro. Primero "desreferencia" el puntero para llegar a la dirección de memoria y luego "selecciona" el campo.

Aplicación en Listas Enlazadas

Cuando gestionas un sistema (por ejemplo, para servicios administrativos o culturales), el operador -> es el que te permite "navegar" por la cadena de datos.

Ejemplo de Selección de Datos:

Si tenemos un puntero llamado aux que apunta a un nodo de la lista:

C

```
printf("ID del servicio: %d", aux->id);
```

Ejemplo de Selección para Enlace:

Para movernos al siguiente elemento de la lista, seleccionamos el puntero siguiente del nodo actual y se lo asignamos al auxiliar:

C

```
aux = aux->siguiente; // "aux ahora apunta a lo que diga el campo siguiente"
```

❖ Operaciones básicos: Inserción

1. Inserción al Inicio (Frente)

Es la operación más rápida ($O(1)$). El nuevo nodo se convierte en la nueva "cabeza" de la lista.

Lógica del algoritmo:

1. Crear el nuevo nodo y asignarle un dato.
2. Hacer que el puntero siguiente del nuevo nodo apunte a donde apunta actualmente la Cabeza.
3. Actualizar la Cabeza para que ahora apunte al nuevo nodo.

---

2. Inserción al Final (Cola)

Si no tienes un puntero de Cola, debes recorrer toda la lista hasta encontrar el nodo cuyo siguiente sea NULL.

Lógica del algoritmo:

1. Crear el nuevo nodo con su campo siguiente en NULL.
2. Si la lista está vacía, el nuevo nodo es la Cabeza.
3. Si no, recorrer la lista con un puntero auxiliar hasta el último nodo.
4. Hacer que el siguiente del último nodo apunte al nuevo nodo.

---

3. Inserción en una Posición Específica (o Lista Ordenada)

Se utiliza para mantener la lista organizada (por ejemplo, por orden alfabético o por ID de servicio administrativo).

Lógica del algoritmo:

1. Localizar el nodo anterior a la posición donde queremos insertar.
2. El siguiente del nuevo nodo debe apuntar al nodo que seguía al "anterior".
3. El siguiente del nodo "anterior" ahora debe apuntar al nuevo nodo.

❖ Operaciones Básicas: Búsqueda

1. Lógica del Algoritmo de Búsqueda

Para realizar una búsqueda de manera segura y eficiente, se siguen estos pasos lógicos:

1. **Inicialización:** Se crea un puntero auxiliar (por ejemplo, actual) y se iguala a la Cabeza de la lista.
2. **Ciclo de Recorrido:** Mientras el puntero actual no sea NULL:
    - Se compara el dato del nodo (actual->dato) con el valor buscado.
    - Si coinciden, la búsqueda ha tenido éxito (se puede devolver el nodo o un mensaje de confirmación).
    - Si no coinciden, el puntero actual avanza al siguiente nodo: actual = actual->siguiente.
3. **Finalización:** Si el ciclo termina y el puntero llega a NULL, significa que el valor no existe en la lista.

---

2. Eficiencia de la Búsqueda

En términos de complejidad algorítmica, la búsqueda en una lista simplemente enlazada tiene una complejidad de $O(n)$.

- **Mejor caso:** El elemento buscado es el primero (la Cabeza). Solo se realiza una comparación.
- **Peor caso:** El elemento es el último o no existe. El algoritmo debe recorrer todos los nodos de la lista.

3. Variantes de la Búsqueda

Dependiendo de la necesidad del sistema (por ejemplo, para la gestión de servicios culturales), la búsqueda puede devolver diferentes resultados:

Tipo de Resultado	**Utilidad**
Booleana (True/False)	Solo para saber si un registro existe (ej. ¿Está registrado este usuario?).
Puntero al	Para poder modificar o leer otros datos de ese

Tipo de Resultado	Utilidad
Nodo	mismo nodo.
Posición (Índice)	Para saber en qué lugar de la fila se encuentra el elemento.
Puntero al Anterior	Crucial si la búsqueda es el paso previo a una eliminación, ya que necesitas saber quién apunta al nodo que vas a borrar.

❖ Operaciones Básicas: Eliminación de elementos

1. Eliminación al Inicio (Cabeza)

Es la forma más sencilla y rápida ($O(1)$). Solo necesitamos que la Cabeza apunte al segundo nodo de la lista.

Lógica del algoritmo:

1. Verificar si la lista está vacía (si es así, no hay nada que borrar).
2. Crear un puntero auxiliar (temporal) que apunte al primer nodo.
3. Mover la Cabeza al siguiente nodo: Cabeza = Cabeza->siguiente.
4. **Liberar la memoria** del nodo al que apunta temporal.

---

2. Eliminación al Final (Cola)

Es más costosa ($O(n)$) porque, en una lista simple, para borrar el último nodo necesitas saber quién es el penúltimo para que su puntero siguiente pase a ser NULL.

Lógica del algoritmo:

1. Recorrer la lista con dos punteros: actual (el que vamos a borrar) y anterior (el que se quedará como último).
2. Al llegar al final, hacer que anterior->siguiente = NULL.
3. Liberar la memoria de actual.

---

3. Eliminación de un Nodo Específico (Búsqueda y Borrado)

Se utiliza cuando queremos borrar un registro puntual (por ejemplo, dar de baja un servicio administrativo o una actividad cultural por su ID).

Lógica del algoritmo:

1. Buscar el nodo que contiene el valor deseado, manteniendo siempre un puntero al nodo anterior.
2. "Saltar" el nodo a eliminar: hacer que el siguiente del nodo anterior apunte al siguiente del nodo que vamos a borrar.
3. Liberar la memoria del nodo seleccionado.

Resumen de Complejidad

Escenario	Complejidad	Notas
Inicio	$O(1)$	Instantáneo. Ideal para Pilas (Stacks).

Escenario	Complejidad	Notas
Final	$O(n)$	Requiere recorrer toda la lista para hallar el penúltimo.
Específico	$O(n)$	El tiempo depende de qué tan lejos esté el nodo a borrar.

❖ Listas doblemente enlazadas y circulares: Declaración

1. Lista Doblemente Enlazada (Doubly Linked List)

En esta estructura, cada nodo tiene **dos enlaces**: uno hacia el nodo siguiente y otro hacia el anterior. Esto permite recorrer la lista en ambos sentidos.

Estructura del Nodo

Para declarar el nodo, necesitamos definir dos punteros del mismo tipo que la estructura:

C:

```
struct NodoDoble {
   int dato;                  // Información
   struct NodoDoble *siguiente; // Puntero al sucesor
   struct NodoDoble *anterior;  // Puntero al predecesor
};
```

Declaración de Control

Es una buena práctica declarar tanto la **cabeza** como la **cola** para aprovechar al máximo la bidireccionalidad:

C:

```
struct NodoDoble *cabeza = NULL;
struct NodoDoble *cola = NULL;
```

---

2. Lista Circular Simplemente Enlazada

La declaración del nodo es **idéntica** a la de una lista simple. La diferencia fundamental no está en la estructura del nodo, sino en **cómo se inicializa y se conecta el último elemento**.

Estructura del Nodo

C:

```
struct NodoCircular {
    int dato;
    struct NodoCircular *siguiente;
};
```

Lógica de Declaración e Inicialización

Al crear el primer nodo, su puntero siguiente no debe ser NULL, sino que debe apuntar **a sí mismo**.

C:

```
struct NodoCircular *inicio = NULL;

// Al insertar el primer nodo 'nuevo':
// nuevo->siguiente = nuevo;
// inicio = nuevo;
```

---

3. Lista Circular Doblemente Enlazada

Es la combinación más completa. Cada nodo tiene dos punteros y no existen finales con NULL.

Estructura del Nodo

Es la misma que la de la lista doblemente enlazada:

C:

```
struct NodoCircDoble {
   int dato;
   struct NodoCircDoble *siguiente;
   struct NodoCircDoble *anterior;
};
```

Conexiones de Control

En la declaración lógica, el siguiente del último apunta al primero, y el anterior del primero apunta al último. Esto crea un anillo perfecto que se puede recorrer en cualquier dirección sin encontrar nunca un fin.

Cuadro Comparativo de Declaración

Tipo de Lista	**Punteros en el struct**	**Inicialización del último siguiente**
Doble	siguiente y anterior	NULL
Circular Simple	siguiente	Cabeza (Inicio)
Circular	siguiente y	Cabeza (y el anterior de la

Tipo de Lista	Punteros en el struct	Inicialización del último siguiente
Doble	anterior	cabeza a la cola)

❖ Listas doblemente enlazadas y circulares: Recorrido

1. Recorrido en Listas Doblemente Enlazadas

Al tener dos punteros (siguiente y anterior), estas listas permiten un desplazamiento en ambos sentidos.

A. Recorrido hacia adelante (Forward)

Es idéntico al de una lista simple. Se parte de la Cabeza y se avanza usando el puntero siguiente hasta encontrar NULL.

- **Lógica:** aux = aux->siguiente;

B. Recorrido hacia atrás (Backward)

Es una de las grandes ventajas de esta estructura. Si tienes un puntero a la Cola (el último nodo), puedes recorrer la lista en orden inverso.

- **Lógica:** aux = aux->anterior;
- **Utilidad:** Ideal para funciones de "Deshacer" o para mostrar registros desde el más reciente al más antiguo.

---

2. Recorrido en Listas Circulares

En las listas circulares, el recorrido presenta un reto: no existe el NULL. Si usas un ciclo convencional, el programa entrará en un bucle infinito.

Estrategia de "Parada"

Para recorrer una lista circular (ya sea simple o doble), la condición de salida no es encontrar el vacío, sino volver al punto de partida.

Pasos lógicos:

1. Verificar si la lista está vacía.
2. Crear un puntero aux que apunte al inicio.
3. Usar un ciclo do-while para asegurar que el código se ejecute al menos una vez antes de comparar si aux ha vuelto a ser igual a inicio.

C:

```
// Ejemplo lógico en C
if (inicio != NULL) {
    aux = inicio;
    do {
        printf("%d ", aux->dato);
        aux = aux->siguiente;
    } while (aux != inicio);
}
```

---

3. Comparación de Recorridos

Tipo de Lista	Sentidos	Condición de Parada	Complejidad

Tipo de Lista	Sentidos	Condición de Parada	Complejidad
Doble	2 (Adelante/Atrás)	aux == NULL	$O(n)$
Circular Simple	1 (Adelante)	aux == inicio (tras la 1ra vez)	$O(n)$
Circular Doble	2 (Adelante/Atrás)	aux == inicio	$O(n)$

❖ Listas doblemente enlazadas y circulares: Inserción

1. Inserción en Listas Doblemente Enlazadas

En este caso, cada vez que insertamos un nodo, debemos conectar **cuatro enlaces** (dos del nuevo nodo y uno de cada nodo vecino).

A. Al Inicio

1. El siguiente del nuevo nodo apunta a la Cabeza.
2. El anterior de la Cabeza actual apunta al nuevo nodo.
3. El anterior del nuevo nodo se pone en NULL.
4. La Cabeza se actualiza al nuevo nodo.

B. Al Final

1. El anterior del nuevo nodo apunta a la Cola.
2. El siguiente de la Cola actual apunta al nuevo nodo.
3. El siguiente del nuevo nodo se pone en NULL.
4. La Cola se actualiza al nuevo nodo.

---

2. Inserción en Listas Circulares (Simples)

Aquí el reto es mantener el "anillo" cerrado. El último nodo siempre debe apuntar al primero.

A. Al Inicio

1. El nuevo nodo apunta a la Cabeza actual.
2. Debes recorrer la lista hasta el último nodo para que su siguiente apunte al nuevo nodo (la nueva cabeza).
3. Actualizar la Cabeza.

B. Al Final

1. El nuevo nodo apunta a la Cabeza.
2. El siguiente del antiguo último nodo apunta al nuevo.

---

3. Inserción en Listas Circulares Dobles

Es la operación más robusta. No requiere recorridos para insertar al final si mantienes un puntero a la Cabeza, ya que el "último" es simplemente Cabeza->anterior.

Pasos para insertar un nuevo nodo:

1. El siguiente del nuevo apunta a la Cabeza.
2. El anterior del nuevo apunta al actual último (Cabeza->anterior).
3. El siguiente del actual último apunta al nuevo.
4. El anterior de la Cabeza apunta al nuevo.

Resumen de Conexiones

Tipo de Lista	Punteros a actualizar por inserción	Complejidad con Punteros de Control
Doble	4 (2 del nuevo, 1 del anterior, 1 del siguiente)	$O(1)$ (en extremos)
Circular Simple	2 (1 del nuevo, 1 del último)	$O(1)$ (si usas puntero al último)
Circular Doble	4 (mantiene el anillo en ambos sentidos)	$O(1)$

- **Listas doblemente enlazadas y circulares: Eliminación de elementos**

1. Eliminación en Listas Doblemente Enlazadas

Al tener punteros hacia adelante (sig) y hacia atrás (ant), la eliminación es muy eficiente porque no necesitas recorrer toda la lista para encontrar al "padre" del nodo que vas a borrar.

Procedimiento para un nodo intermedio:

1. **Localización:** Identificar el nodo X a eliminar.
2. **Re-direccionamiento:**
    - El siguiente del nodo **anterior** a X debe apuntar al que sigue de X.

    - El anterior del nodo **posterior** a X debe apuntar al que estaba antes de X.
3. **Liberación:** Usar free(X) para devolver la memoria al sistema.

Caso Especial: Si eliminas la **Cabeza**, recuerda actualizar el puntero inicio al segundo nodo y poner su campo anterior en NULL.

---

2. Eliminación en Listas Circulares (Simples)

El desafío aquí es que el último nodo siempre debe quedar apuntando al primero. Si borras el primero o el último, el "cierre" del anillo debe actualizarse.

Procedimiento:

1. **Búsqueda:** Localizar el nodo a borrar y su **antecesor**.
2. **Puente:** El antecesor debe apuntar al nodo que seguía al que estamos borrando.
3. **Actualización de Inicio:** Si el nodo borrado era la Cabeza, el puntero de control de la lista ahora debe apuntar al siguiente, y el último nodo de la lista debe actualizar su enlace para apuntar a este nuevo inicio.

---

3. Eliminación en Listas Circulares Dobles

Es la estructura más robusta. Gracias a que es doble, puedes llegar al nodo anterior y al siguiente instantáneamente desde el nodo que quieres borrar.

Lógica de "Salto":

- NodoAnterior->siguiente = NodoEliminar->siguiente;
- NodoSiguiente->anterior = NodoEliminar->anterior;

Incluso si solo hay un nodo en la lista, al eliminarlo, simplemente debes asegurar que tu puntero de control (inicio) vuelva a ser NULL.

Comparación de Complejidad en Eliminación

Tipo de Lista	**Localización**	**Re-direccionamiento**	**Complejidad Total**
Doble	$O(n)$	$O(1)$	$O(n)$
Circular Simple	$O(n)$	$O(1)$	$O(n)$
Circular Doble	$O(n)$	$O(1)$	$O(n)$

➢ **Recursividad**

❖ **Fundamentos Teóricos:**

- **Definición**

La recursividad es un concepto fundamental en informática y matemáticas que ocurre cuando una función se define en términos de sí misma. En programación, una función recursiva es aquella que se llama a sí misma para resolver un problema de menor

tamaño, repitiendo este proceso hasta llegar a una solución simple y directa.

Es una alternativa a la iteración (bucles for o while) y es especialmente útil para trabajar con estructuras de datos complejas como árboles o, en tu caso, para recorrer ciertas configuraciones de listas enlazadas.

---

1. Los Dos Pilares de la Recursividad

Para que una función recursiva sea correcta y no cause un error de desbordamiento de memoria (*stack overflow*), debe cumplir con dos reglas estrictas:

- **Caso Base (Condición de Salida):** Es la solución más simple del problema que no requiere más llamadas recursivas. Indica cuándo debe detenerse la función.
- **Caso Recursivo:** Es la parte donde la función se llama a sí misma, pero con un argumento "más pequeño" o más cercano al caso base.

---

2. ¿Cómo funciona internamente? (La Pila de Llamadas)

Cuando una función se llama a sí misma, el sistema operativo no "reinicia" la función, sino que coloca la ejecución actual en una Pila de Llamadas (*Stack*). Cada nueva llamada se apila sobre la anterior. Cuando se alcanza el caso base, las funciones empiezan a resolverse y a "desapilarse" una por una, devolviendo el resultado a la función que la llamó.

- **Ámbito de aplicación**

1. Estructuras de Datos Jerárquicas (Árboles y Grafos)

Este es el ámbito donde la recursividad brilla por excelencia. Estructuras como los árboles binarios (usados en bases de datos) o los grafos (usados en mapas y redes sociales) son recursivas por naturaleza: un árbol es un nodo que tiene otros "sub-árboles" debajo.

- **Búsqueda en directorios:** Cuando tu computadora busca un archivo en una carpeta, usa recursividad para entrar en una subcarpeta, luego en otra, y así sucesivamente.
- **Recorridos de árboles:** Operaciones como *Inorden*, *Preorden* y *Postorden* se escriben en 3 o 4 líneas de código recursivo, mientras que de forma iterativa serían muy complejas.

---

2. Algoritmos de "Divide y Vencerás" (Divide and Conquer)

Muchos de los algoritmos de ordenamiento más rápidos del mundo utilizan este enfoque. Consiste en dividir un conjunto de datos en dos mitades, resolver cada mitad recursivamente y luego combinarlas.

- **Merge Sort (Ordenamiento por mezcla):** Divide una lista hasta que cada elemento está solo y luego los mezcla ordenadamente.
- **Quick Sort (Ordenamiento rápido):** Elige un "pivote" y organiza los elementos mayores y menores a él de forma recursiva.

---

3. Procesamiento de Listas Enlazadas

Aunque las listas se pueden recorrer con un simple bucle while, la recursividad permite realizar tareas complejas de forma más elegante:

- **Impresión inversa:** Imprimir una lista desde el último nodo hasta el primero es trivial con recursividad (imprimes *después* de la llamada recursiva).
- **Copia de listas:** Crear una copia exacta de una estructura dinámica.

---

4. Resolución de Problemas Matemáticos y Combinatorios

Existen problemas que tienen una definición matemática intrínsecamente recursiva:

- **Cálculo de potencias y factoriales.**
- **Sucesión de Fibonacci.**
- **Torres de Hanói:** Un puzzle clásico que es prácticamente imposible de resolver de forma eficiente sin recursividad.
- **Fractales:** En computación gráfica, para generar figuras que se repiten a sí mismas a diferentes escalas (como nubes, montañas o costas en videojuegos).

---

5. Análisis de Lenguajes (Parsing)

Los compiladores (los programas que traducen tu código C o JavaScript a lenguaje de máquina) utilizan recursividad para entender la gramática de lo que escribes.

- **Evaluación de expresiones:** Para resolver una operación como (5 + (3 * 2)), el sistema evalúa primero el paréntesis interno recursivamente antes de sumar el 5.

- **Utilidad**

La **utilidad** de la recursividad radica en su capacidad para simplificar problemas que, de ser resueltos con bucles tradicionales (for o while), requerirían un código mucho más extenso, complejo y difícil de mantener. Es una herramienta de "elegancia algorítmica".

---

1. Simplificación de Código (Legibilidad)

Existen algoritmos que son intrínsecamente recursivos. Escribirlos de forma iterativa obliga al programador a gestionar manualmente una estructura de datos llamada Pila (*Stack*), lo que ensucia el código. La recursividad permite que el propio sistema se encargue de esa gestión.

- **Ejemplo:** Recorrer todas las carpetas y subcarpetas de un disco duro. Con recursividad, solo necesitas una función que se llame a sí misma cada vez que encuentra una carpeta nueva.

---

2. Resolución de Problemas de "Naturaleza Repetitiva"

Muchos problemas matemáticos y de lógica se definen mejor mediante la recursividad. Su utilidad aquí es que el código refleja casi exactamente la definición teórica del problema.

- **Torres de Hanói:** Un rompecabezas clásico cuya solución iterativa es extremadamente confusa, pero cuya solución recursiva solo requiere unas 5 líneas de código.
- **Cálculo de Fractales:** Utilizado en computación gráfica para generar paisajes, nubes o texturas naturales de forma infinita.

---

3. Procesamiento Eficiente de Estructuras Dinámicas

En el manejo de listas enlazadas y árboles, la recursividad es sumamente útil para realizar tareas que requieren "recordar" el camino de regreso.

- **Impresión Inversa:** Si quieres imprimir una lista desde el último nodo hasta el primero, la recursividad permite llegar al final y empezar a imprimir mientras la función "regresa" (se desapila).
- **Búsqueda en Árboles Binarios:** Permite descartar la mitad de los datos en cada paso, haciendo que la búsqueda sea increíblemente rápida en bases de datos masivas.

---

4. Algoritmos de Ordenamiento y Búsqueda

La recursividad es el motor de los algoritmos de ordenamiento más rápidos que existen hoy en día, utilizados en motores de búsqueda y redes sociales.

- **QuickSort y MergeSort:** Utilizan la estrategia de "Divide y Vencerás". Dividen una lista desordenada en partes pequeñas, las ordenan recursivamente y luego las combinan. Sin recursividad, estos algoritmos perderían su claridad y eficiencia de implementación.

- **Ventajas y Desventajas de la recursividad**

1. Ventajas de la Recursividad

- **Simplicidad y Elegancia:** Muchos algoritmos son naturalmente recursivos (como el recorrido de árboles o el ordenamiento *Merge Sort*). Implementarlos de forma recursiva requiere menos líneas de código y es más fácil de leer.

- **Facilidad para Estructuras Complejas:** Es la herramienta ideal para trabajar con estructuras que no son lineales, como los directorios de archivos o árboles genealógicos, donde cada nodo puede tener múltiples hijos.
- **División del Problema:** Aplica perfectamente la filosofía de "Divide y Vencerás", permitiendo resolver problemas grandes dividiéndolos en subproblemas idénticos más pequeños.
- **Mantenimiento:** Al ser un código más corto y apegado a la lógica matemática, suele ser más sencillo de depurar y mantener por otros programadores en el futuro.

---

2. Desventajas de la Recursividad

- **Consumo de Memoria (Stack Overflow):** Cada llamada recursiva crea un nuevo "marco" en la Pila de Llamadas **(Stack)**. Si la recursividad es muy profunda (por ejemplo, recorrer una lista de 100,000 nodos), el programa puede agotar la memoria y colapsar.
- **Rendimiento (Velocidad):** Las llamadas a funciones tienen un costo de tiempo adicional (gestión de punteros, retorno de valores y salto de instrucciones). Un bucle while suele ser más rápido para tareas simples.
- **Dificultad de Rastreo:** Para quienes están iniciando, seguir el flujo de una función que se llama a sí misma múltiples veces puede ser confuso, especialmente cuando hay múltiples llamadas en una misma línea.
- **Cálculos Redundantes:** Si no se programa con cuidado (por ejemplo, en la serie de Fibonacci básica), la recursividad puede terminar calculando el mismo valor miles de veces innecesariamente.

- **Diseño y escritura de programas recursivos**

El diseño y la escritura de programas recursivos requiere un cambio de mentalidad: en lugar de pensar en cómo repetir una tarea (iteración), debemos pensar en cómo definir el problema en términos de una versión más pequeña de sí mismo.

---

1. Identificar el "Caso Base" (La Salida)

Es la parte más crítica. El caso base es la instancia del problema que es tan simple que se puede resolver directamente sin más recursividad. Sin un caso base, el programa se ejecutaría infinitamente.

- **En listas:** Generalmente es cuando el puntero es NULL.
- En cálculos: Generalmente es cuando un contador llega a 0 o 1.
- **En búsquedas:** Cuando se encuentra el elemento o se acaba el espacio de búsqueda.

2. Definir la "Relación de Recurrencia"

Es la lógica que divide el problema. Debes determinar cómo el problema de tamaño N se relaciona con el problema de tamaño $N-1$.

Regla de Oro: Cada llamada recursiva debe acercar el argumento un paso más hacia el Caso Base.

---

3. La Estructura del Código

Un programa recursivo estándar sigue casi siempre este esquema en lenguajes como C o JavaScript:

C:
```
TipoDeDato funcionRecursiva(parametros) {
```

```
    // 1. SECCIÓN DE CONTROL (Caso Base)
    if (condicion_de_parada) {
        return resultado_simple;
    }

    // 2. SECCIÓN DE PROCESAMIENTO (Opcional)
    // Aquí haces algo con los datos antes de la llamada

    // 3. LLAMADA RECURSIVA (Acercamiento al caso base)
    return funcionRecursiva(parametros_reducidos);
}
```

---

4. Escritura: ¿Antes o Después de la Llamada?

El orden en que escribes las instrucciones respecto a la llamada recursiva cambia totalmente el comportamiento del programa:

- Pre-orden (Antes): Si procesas los datos *antes* de la llamada, trabajas mientras la pila crece (de la cabeza al final).
- Post-orden (Después): Si procesas los datos *después* de la llamada, trabajas mientras la pila se libera (del final hacia la cabeza).

Ejemplo Práctico: Contar Nodos en una Lista

Si aplicamos esto al desarrollo de un sistema (como el de gestión cultural que mencionas), podrías necesitar contar cuántos registros hay en una lista dinámica:

1. Caso Base: Si la lista es NULL, hay 0 nodos.
2. Relación: El total de nodos es 1 (el actual) más el total de nodos del resto de la lista.

C
```
int contarNodos(struct Nodo *h) {
    if (h == NULL) {       // Caso Base
```

```
        return 0;
    } else {            // Relación de recurrencia
        return 1 + contarNodos(h->siguiente);
    }
}
```

➢ Introducción a las Estructuras de Datos Dinamicas Avanzadas; Pilas, Colas y Arboles

- **Definición**

Las estructuras de datos dinámicas avanzadas son formas de organizar la información en la memoria de una computadora de manera que su tamaño pueda crecer o reducirse durante la ejecución del programa. A diferencia de las estructuras estáticas (como los arreglos básicos), estas no reservan un bloque fijo de memoria, sino que solicitan espacio a medida que se necesita mediante el uso de punteros y nodos.

Entrar en el terreno de las estructuras avanzadas implica pasar de una organización lineal simple a estructuras con reglas de acceso específicas o jerarquías complejas.

---

1. Pilas (Stacks)

Una pila es una estructura de datos lineal que sigue el principio LIFO (*Last In, First Out*), es decir, el último elemento en entrar es el primero en salir.

- **Concepto:** Imagina una pila de platos; solo puedes añadir un plato arriba o quitar el que está en la cima.
- **Operaciones Clave:**
    - **Push:** Insertar un elemento en la cima.
    - **Pop:** Retirar el elemento de la cima.

    - **Peek/Top:** Observar el elemento superior sin retirarlo.
- **Utilidad:** Gestión de llamadas a funciones (recursividad), deshacer/rehacer en editores de texto y validación de paréntesis en compiladores.

---

2. Colas (Queues)

Una cola es una estructura lineal que sigue el principio **FIFO** (*First In, First Out*), donde el primer elemento en entrar es el primero en ser atendido.

- **Concepto:** Es idéntica a una fila en un banco o en la taquilla de la **Casa de la Cultura**. El primero que llega es el primero que sale.
- **Operaciones Clave:**
    - **Enqueue:** Insertar al final de la cola.
    - **Dequeue:** Eliminar el elemento que está al frente.
- **Utilidad:** Gestión de procesos en sistemas operativos, colas de impresión y buffers de datos en streaming.

---

3. Árboles (Trees)

Los árboles son estructuras de datos **no lineales** y **jerárquicas**. A diferencia de las pilas y colas, un nodo en un árbol puede conectarse con varios nodos "hijos".

- **Concepto:** Se asemeja a un organigrama o a la estructura de carpetas de un sistema operativo. Existe un nodo principal llamado **Raíz**, y de él se desprenden ramas hasta llegar a las **Hojas** (nodos sin hijos).
- **Árbol Binario:** Es el tipo más común, donde cada nodo puede tener como máximo dos hijos (izquierdo y derecho).
- **Utilidad:** Representación de jerarquías administrativas, bases de datos de búsqueda rápida (árboles AVL o B-Trees) y estructuras de decisión en inteligencia artificial.

- **Especificaciones**

1. Especificaciones de la Pila (Stack)

Se rige por la disciplina de acceso LIFO (*Last In, First Out*).

- **Puntero de Cima (Top):** Es la única variable de control. Almacena la dirección del último nodo insertado.
- **Capacidad:** En una implementación dinámica (con nodos), la capacidad es teóricamente infinita (limitada solo por la RAM). En una implementación estática (arreglos), existe el riesgo de **Stack Overflow** (desbordamiento).
- **Condición de Vacío:** Una pila está vacía cuando Top == NULL.
- **Acceso:** No se permite el acceso aleatorio. Para llegar al tercer elemento, primero debes extraer los dos de arriba.

---

2. Especificaciones de la Cola (Queue)

Se rige por la disciplina de acceso FIFO (*First In, First Out*).

- **Doble Puntero de Control:** * **Frente (Front):** De donde se extraen los elementos.
    - **Final (Rear/Back):** Por donde se insertan los nuevos elementos.
- **Variantes Específicas:**
    - **Colas Circulares:** El final se conecta con el principio para reutilizar espacios de memoria vacíos.
    - **Colas de Prioridad:** Los elementos no salen por orden de llegada, sino por un valor de importancia (ej. atención de emergencias médicas).
- **Condición de Vacío:** Front == NULL.

---

3. Especificaciones de los Árboles (Trees)

Al ser una estructura no lineal, sus especificaciones definen la jerarquía y la distancia entre datos.

- **Grado de un Nodo:** El número de hijos que tiene un nodo específico.
- **Altura del Árbol:** El número máximo de niveles desde la raíz hasta la hoja más lejana.
- **Nivel:** La distancia de un nodo respecto a la raíz (la raíz está en el nivel 0).
- **Equilibrio (Balance):** Un árbol está balanceado si la diferencia de altura entre sus subárboles izquierdo y derecho es mínima (máximo 1). Esto es vital para que las búsquedas sean rápidas ($O(\log n)$).
- **Tipos de Nodos:**
  - **Raíz:** El nodo superior sin padre.
  - **Padre/Hijo:** Relación de conexión directa descendente.
  - **Hoja:** Nodos terminales (sin hijos).

- **Tipos**

1. Tipos de Pilas (Stacks)

Aunque la pila lógica es una sola, su implementación varía:

- **Pila Estática:** Utiliza un arreglo de tamaño fijo. Es rápida pero arriesgada si se llena (desbordamiento).
- **Pila Dinámica:** Utiliza nodos y punteros. Es la más usada en sistemas modernos porque crece según la necesidad de la memoria RAM.

2. Tipos de Colas (Queues)

Las colas tienen variantes muy útiles para la administración pública y cultural:

- **Cola Simple:** Inserción al final y extracción al frente.
- **Cola Circular:** El último nodo se conecta con el primero, optimizando el uso de memoria en buffers.
- **Cola de Prioridad:** Los elementos tienen un "valor de importancia". Por ejemplo, en un sistema de trámites, un usuario con discapacidad podría tener prioridad 1 y salir antes que otros que llegaron primero.
- **Bicola (Deque):** Permite insertar y extraer elementos por ambos extremos (frente y final).

---

3. Tipos de Árboles (Trees)

Los árboles se clasifican según su estructura y reglas de llenado:

- **Árbol Binario:** Cada nodo tiene máximo 2 hijos.
- **Árbol Binario de Búsqueda (BST):** Los valores menores van a la izquierda y los mayores a la derecha. Es el tipo ideal para buscar expedientes de artistas o ciudadanos rápidamente.
- **Árboles Balanceados (AVL o Rojo-Negro):** Árboles que se reorganizan solos para que ninguna rama sea mucho más larga que otra, garantizando que la búsqueda siempre sea veloz.
- **Árboles N-arios:** Nodos que pueden tener muchos hijos (útil para representar organigramas administrativos complejos).

- **Funcionalidades e implementación básica**

  1. Pilas (Stacks): Implementación Dinámica
  La funcionalidad principal es el **Push** (apilar) y **Pop** (desapilar). En un sistema de gestión, esto sirve para controlar el flujo de pantallas o "atrás" en un menú.
  Estructura y Push:

C:
```
struct Nodo {
    int dato;
    struct Nodo *sig;
};

void push(struct Nodo **top, int valor) {
    struct Nodo *nuevo = malloc(sizeof(struct Nodo));
    nuevo->dato = valor;
    nuevo->sig = *top; // El nuevo apunta al que era la cima
    *top = nuevo;      // La cima ahora es el nuevo
}
```

---

2. Colas (Queues): Gestión de Turnos

La funcionalidad clave es el **Enqueue** (encolar) y **Dequeue** (desencolar). Para el proyecto en la **Casa de la Cultura**, esto es lo que gestionaría la fila de ciudadanos esperando un trámite.

Lógica de Enqueue (Insertar al final):

C:
```
void enqueue(struct Nodo **frente, struct Nodo **final, int
valor){
    struct Nodo *nuevo = malloc(sizeof(struct Nodo));
    nuevo->dato = valor;
    nuevo->sig = NULL;
    if (*frente == NULL) {
        *frente = *final = nuevo;
    } else {
        (*final)->sig = nuevo;
        *final = nuevo;
    }
}
```

---

3. Árboles Binarios de Búsqueda (BST): Implementación
Su funcionalidad estrella es la Inserción Ordenada, que permite que las búsquedas futuras sean extremadamente rápidas. Es la base para organizar el inventario cultural o el censo de artistas.

Inserción Recursiva:

C:
```
struct NodoArbol {
    int info;
    struct NodoArbol *izq, *der;
};

struct NodoArbol* insertar(struct NodoArbol *raiz, int valor) {
    if (raiz == NULL) {
        struct NodoArbol *nuevo = malloc(sizeof(struct
NodoArbol));
        nuevo->info = valor;
        nuevo->izq = nuevo->der = NULL;
        return nuevo;
    }
    if (valor < raiz->info)
        raiz->izq = insertar(raiz->izq, valor);
    else
        raiz->der = insertar(raiz->der, valor);
    return raiz;
}
```